第6章　树和二叉树 作业（共11题）

6.5　6.8　6.17　6.25　6.26　6.29　6.42　6.43　6.47　6.49　6.65

6.4 树和森林

6.4.1 树和森林与二叉树的转换

6.4.2 树和森林的存储方式

6.4.3 树和森林的遍历

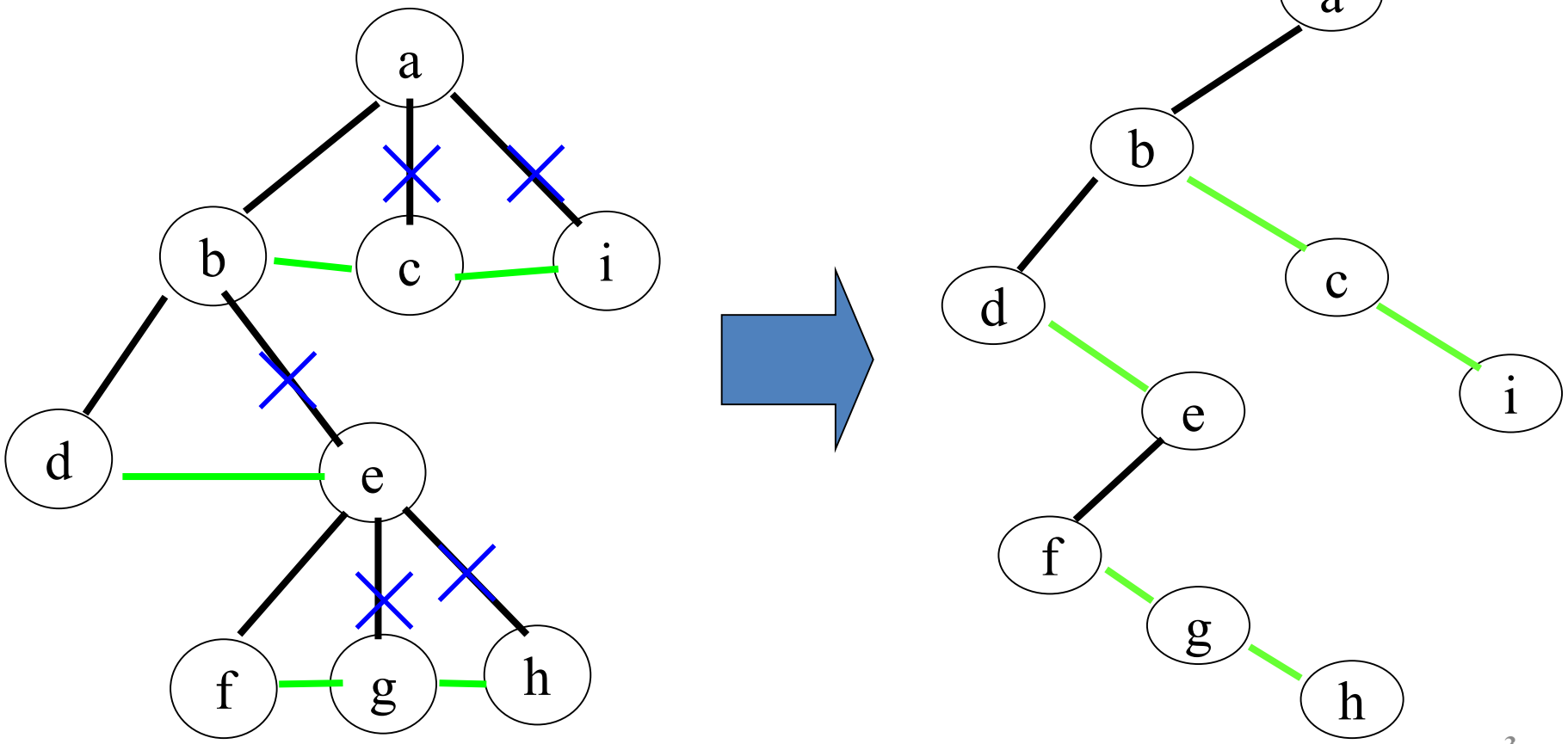
6.4.1 树和森林与二叉树的转换

左孩子—右兄弟表示法

回顾1：树如何转为二叉树?

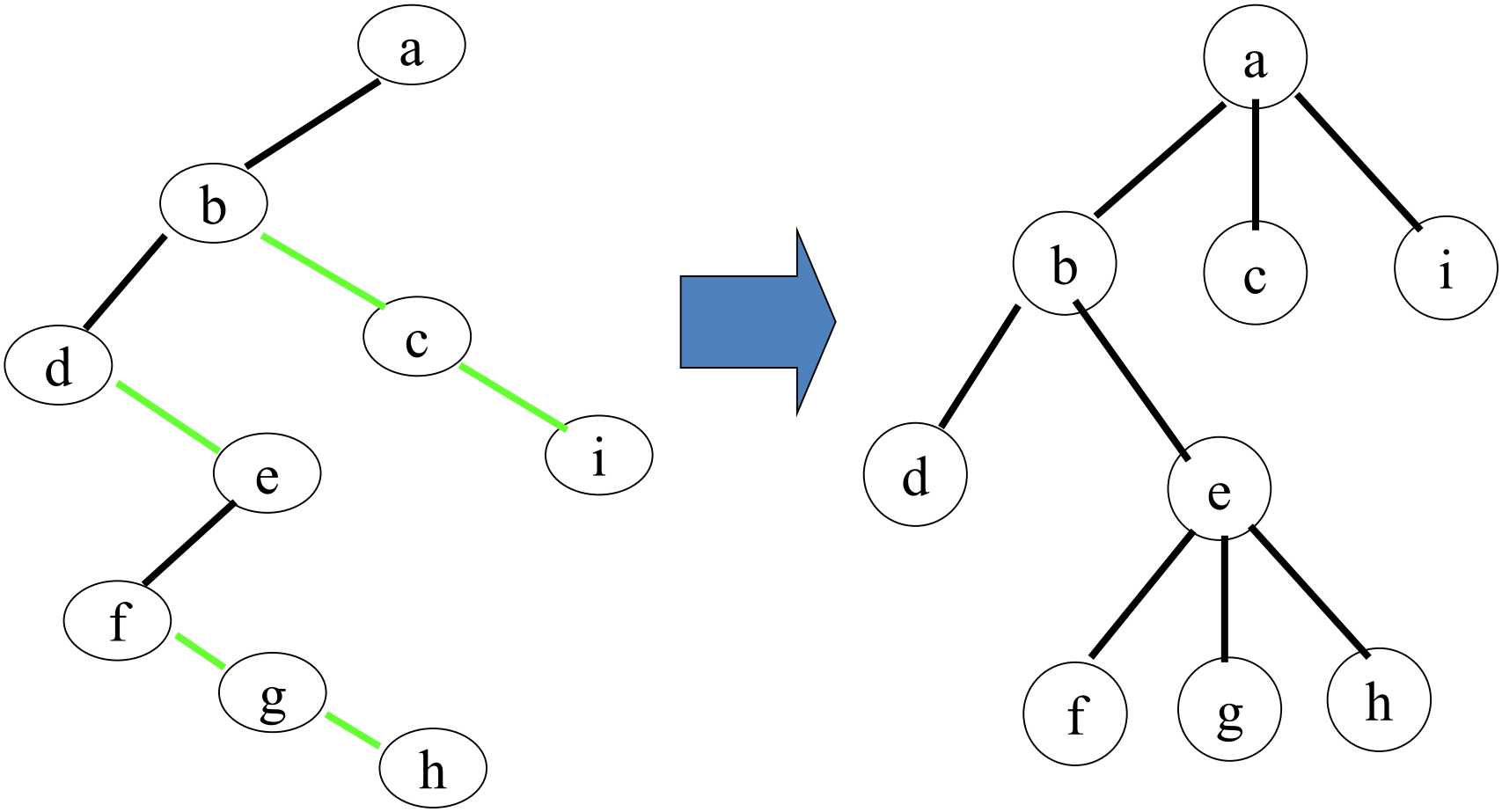
方法：加线—抹线—旋转

兄弟相连　长兄为父　孩子靠左



回顾2：二叉树怎样还原为树？

要点：逆操作，把所有右孩子变为兄弟！



讨论1：森林如何转为二叉树？

即$F=\{T_1, T_2, \ldots, T_m\} \Longleftrightarrow B=\{root, LB, RB\}$

法一：

① 各森林先各自转为二叉树；

② 依次连到前一个二叉树的右子树上。

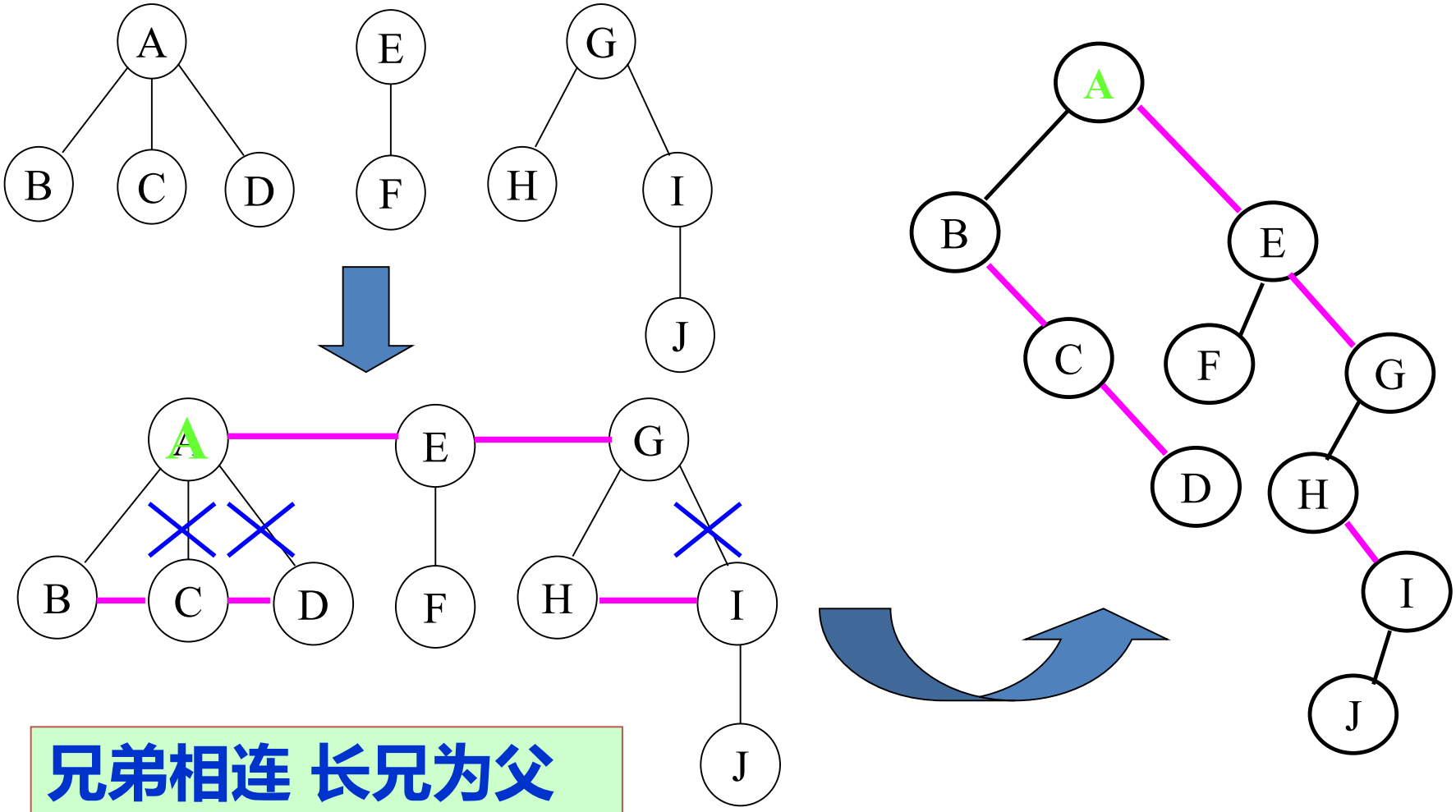
法二：森林直接变兄弟，再转为二叉树

(参见教材P138图6.17，两种方法都有转换示意图)

法一和法二得到的二叉树是完全相同的、惟一的。

森林转二叉树举例：

（用法二，森林直接变兄弟，再转为二叉树）

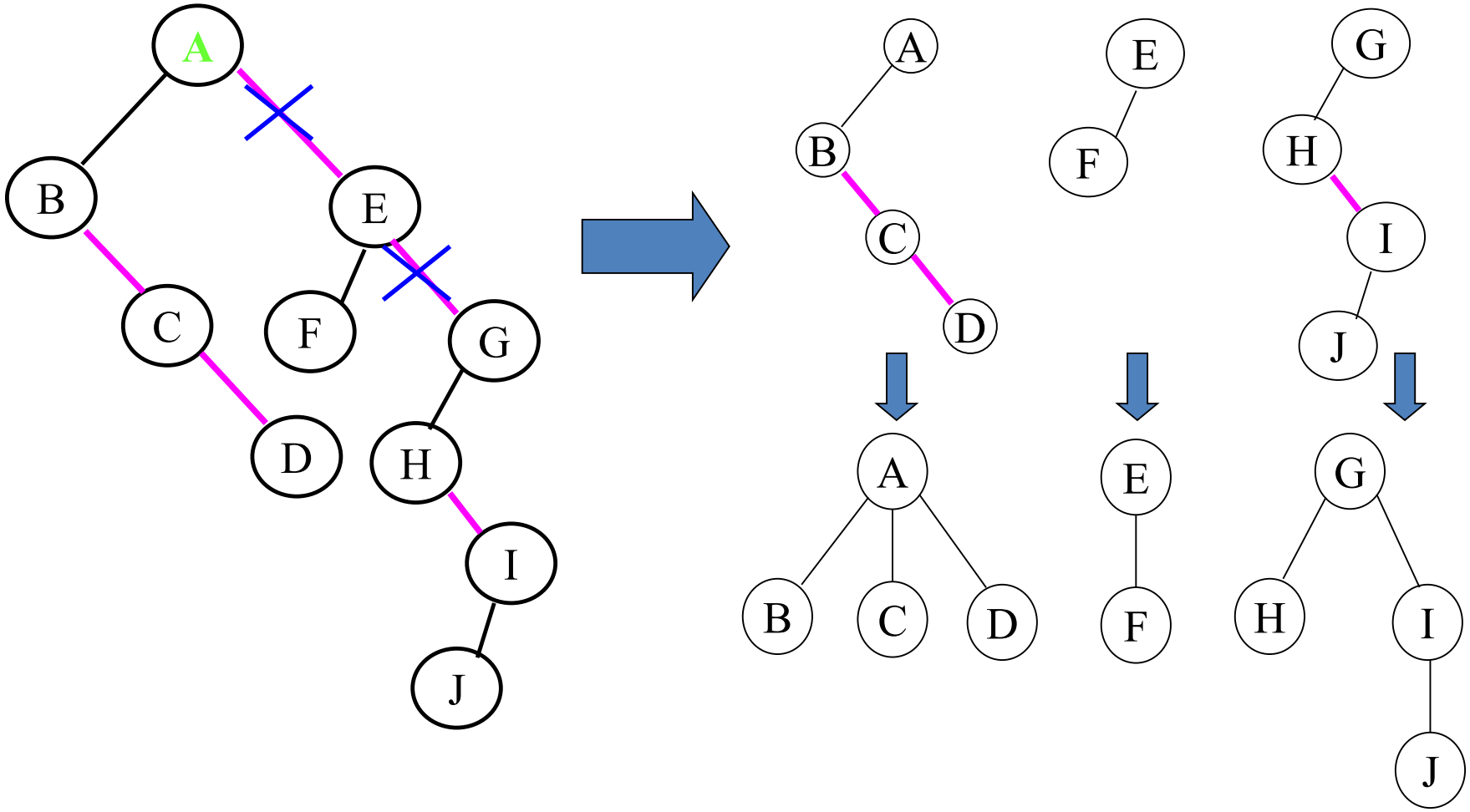


兄弟相连 长兄为父
头树为根 孩子靠左

讨论2：二叉树如何还原为森林？

即B={root, LB, RB}⟷F={T_1, T_2, …,T_m}

要点：把最右边的子树变为森林，其余右子树变为兄弟



6.4.2 树和森林的存储方式

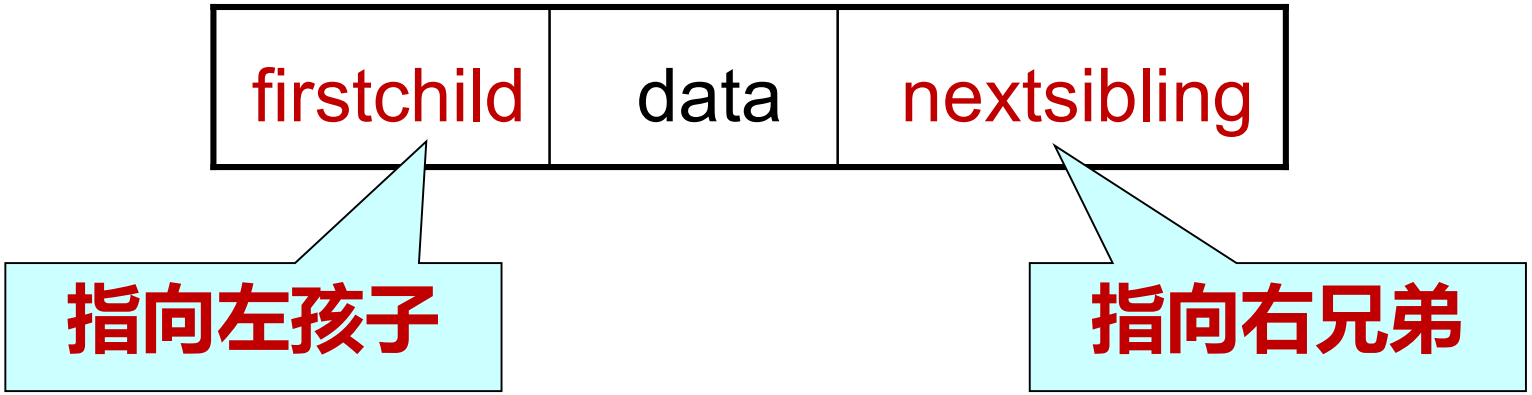
树有三种常用存储方式：

①双亲表示法　②孩子表示法　③孩子—兄弟表示法

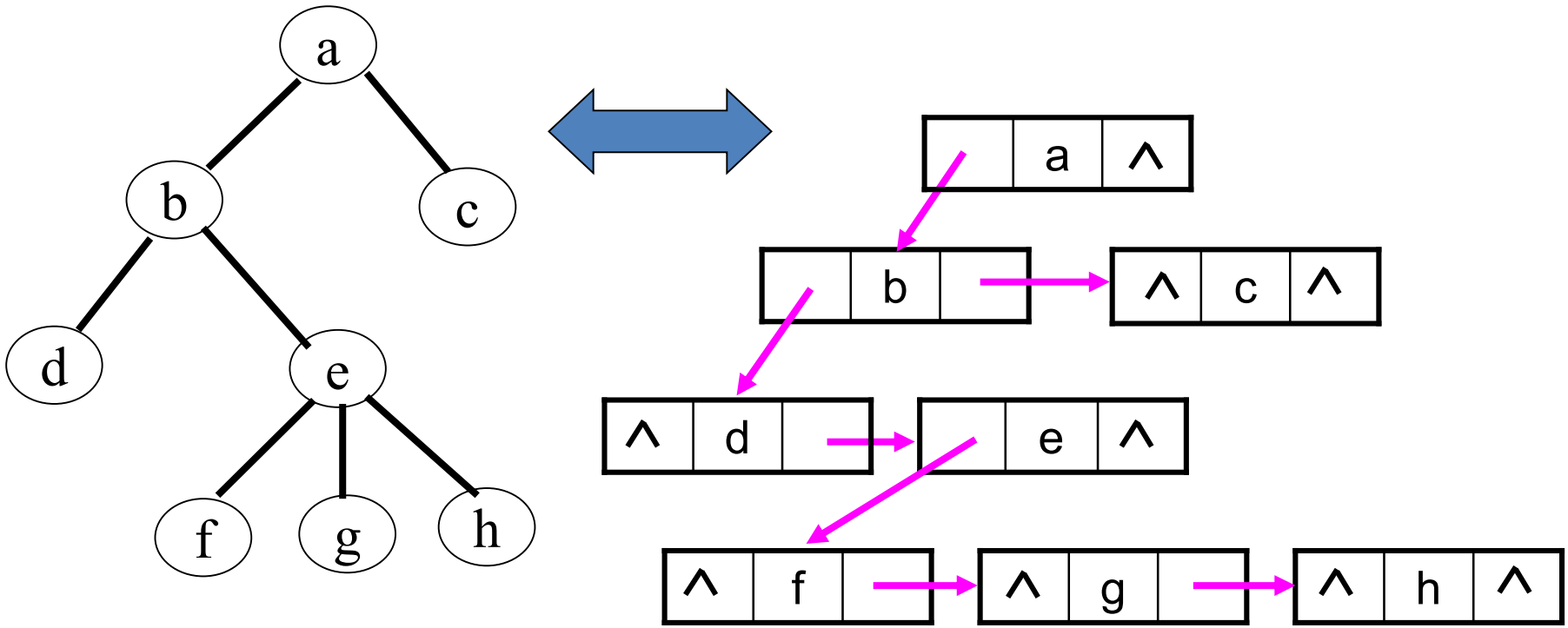
问：树→二叉树的“连线—抹线—旋转” 如何由计算机自动实现?

答：用“左孩子右兄弟”表示法来存储即可。

存储的过程就是树转换为二叉树的过程！



例如：



6.4.3 树和森林的遍历

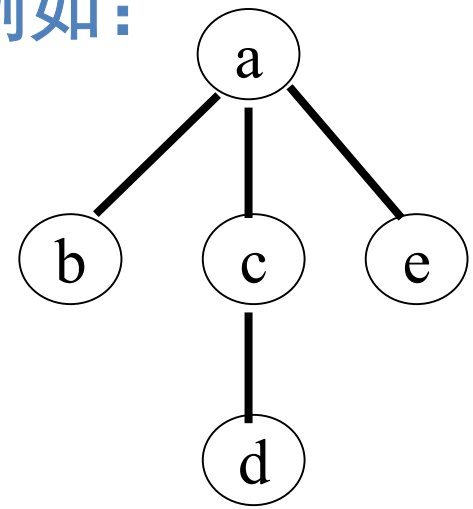
树的遍历
- 深度优先遍历（先根、后根）
- 广度优先遍历（层次）

树没有中序遍历（因子树不分左右）

- **先根遍历**
  - 访问根结点；
  - 依次先根遍历根结点的每棵子树。

- **后根遍历**
  - 依次后根遍历根结点的每棵子树；
  - 访问根结点。

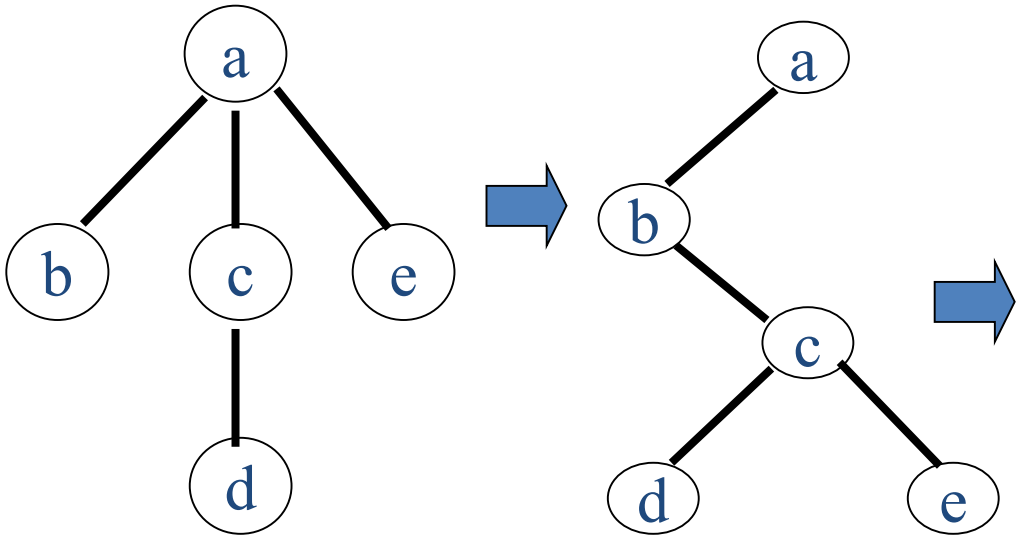
例如：



先根序列： a b c d e

后根序列： b d c e a

讨论：树若采用“先转换，后遍历”方式，结果是否一样?



先序遍历：a b c d e

中序遍历：b d c e a

后序遍历：d e c b a

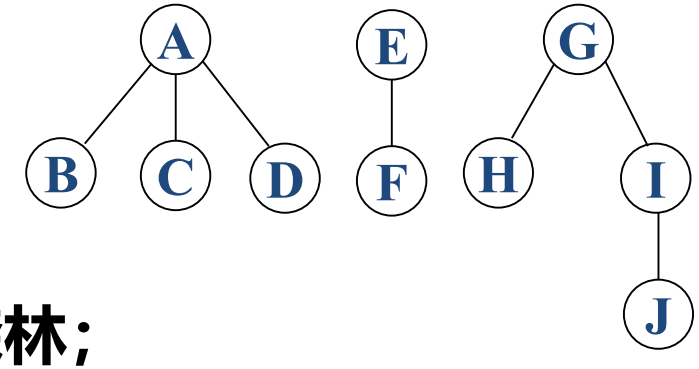
树的先根序列：a b c d e
树的后根序列：b d c e a

结论：

1. 树的先根遍历与二叉树的先序遍历相同；
2. 树的后根遍历相当于二叉树的中序遍历；
3. 树没有中序遍历，因为子树无左右之分。

森林的遍历 { 深度优先遍历（先序、中序）
广度优先遍历（层次）

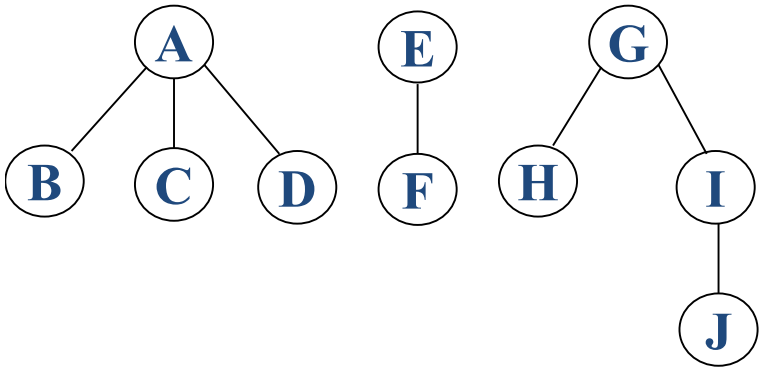
为何有中序?



- **先序遍历**
  - 若森林为空，返回；
  - 访问森林中第一棵树的根结点；
  - 先根遍历第一棵树的根结点的子树森林；
  - 先根遍历除去第一棵树之后剩余的树构成的森林。

- **中序遍历**
  - 若森林为空，返回；
  - 中根遍历森林中第一棵树的根结点的子树森林；
  - 访问第一棵树的根结点；
  - 中根遍历除去第一棵树之后剩余的树构成的森林。

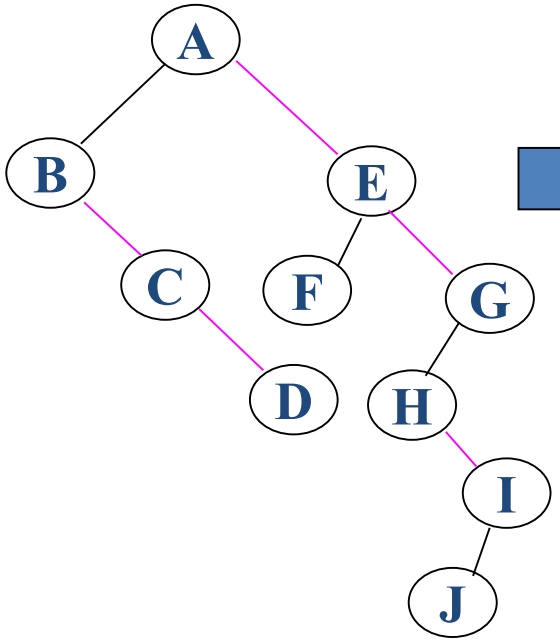
例如：



先序序列：A B C D E F G H I J

中序序列：B C D A F E H J I G

讨论：若采用“先转换，后遍历”方式，结果是否相同?



先序序列：A B C D E F G H I J

中序序列：B C D A F E H J I G

结论：森林的先序和中序遍历在两种方式下的结果相同。

6.5A 二叉树的典型应用（仅介绍）

平衡树——特点：所有结点左右子树深度差≤1

排序树——特点：所有结点“左小右大”

字典树——由字符串构成的二叉排序树

判定树——特点：分支查找树（例如12个球如何只称3次便分出轻重）

带权树——特点：路径带权值（例如长度）

最优树——是带权路径长度最短的树，又称 Huffman树，用途之一是通信中的压缩编码。

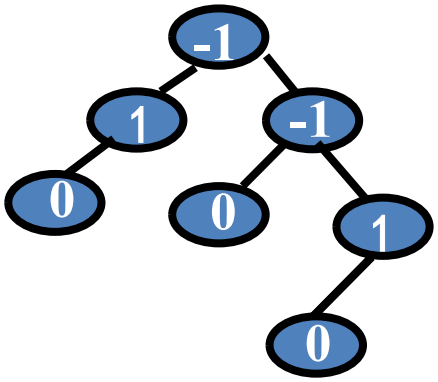
什么是平衡二叉树（又称AVL 树）？

性质： 所有结点左、右子树深度之差的绝对值 ≤ 1

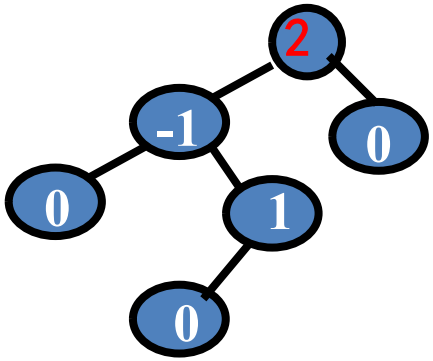
若定义结点的“平衡因子” BF = 左子树深度 – 右子树深度

则：平衡二叉树中所有结点的BF ∈ [-1, 0, 1]

例：判断下列二叉树是否AVL树？



(a) 平衡树



(b) 不平衡树

什么是二叉排序树？

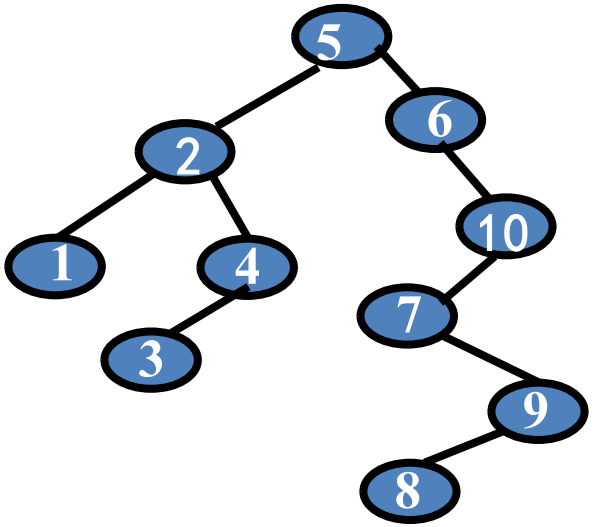
-----或是一棵空树；或者是具有如下性质的非空二叉树：

（1）左子树的所有结点均小于根的值；

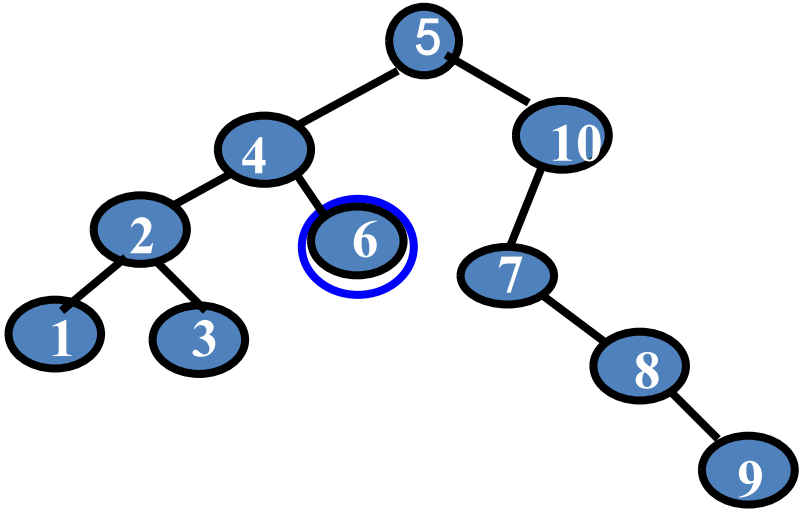
（2）右子树的所有结点均大于根的值；

（3）它的左右子树也分别为二叉排序树。

例：下列2种图形中，哪个不是二叉排序树？



（a）



（b）

想一想：对它中序遍历之后是什么效果?

什么是判定树？

举例： **12**个球如何用天平只称**3**次便分出轻重？

分析：

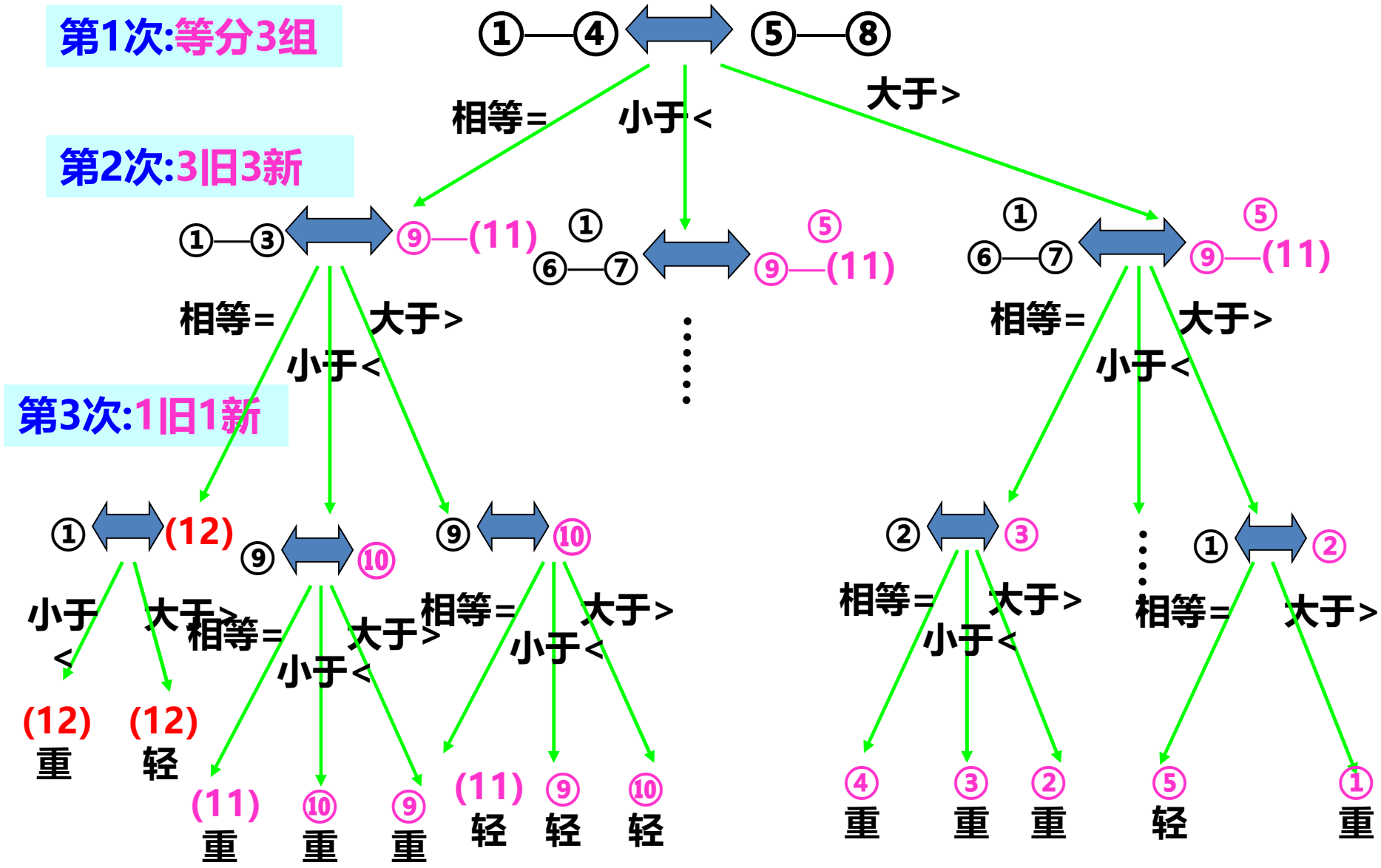
12个球中必有一个非轻即重，即共有24种“次品”的可能性。每次天平称重的结果有3种，连称3次应该得到的结果有$3^3=27$种。

说明仅用3次就能找出次品的可能性是存在的。

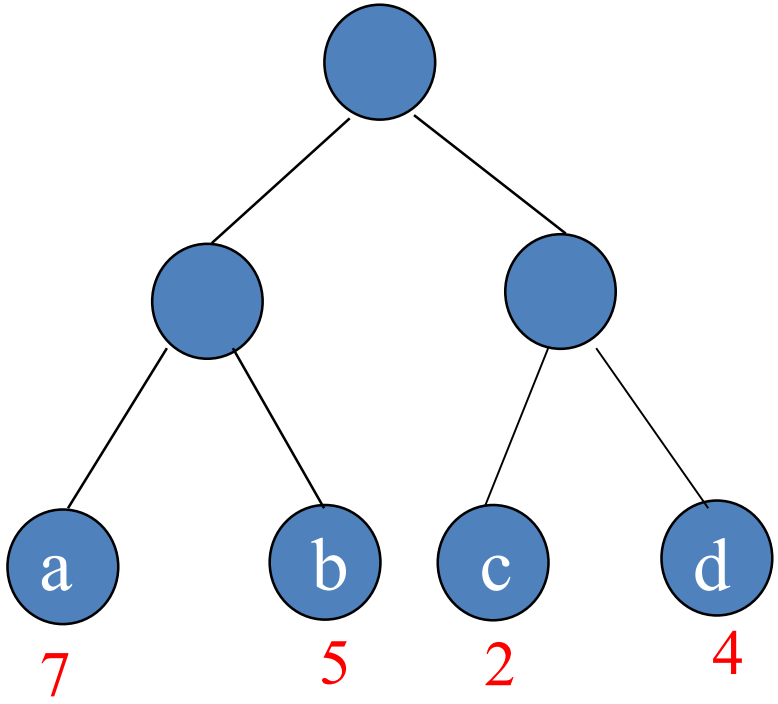
思路：

首先，将12个球分三组，每组4个，任意取两组称。会有3种结果：平衡、或左>右、或左<右。

其次，一定要利用已经称过的那些结论；即充分利用“旧球”的标准性作为参考。


第1次:等分3组
第2次:3旧3新
第3次:1旧1新
①—④ ⟺ ⑤—⑧
相等=
小于<
大于>
①—③ ⟺ ⑨—(11)
①⑥—⑦ ⟺ ⑤⑨—(11)
①⑥—⑦ ⟺ ⑤⑨—(11)
①(12)
⑨⟺⑩
⑨⟺⑩
②③
①②
(12) 重
(12) 轻
(11) 重
⑩ 重
⑨ 重
(11) 轻
⑨ 轻
⑩ 轻
④ 重
③ 重
② 重
⑤ 轻
① 重

什么是带权树？ **即叶子带有权值。例如：**



如果是带权路径长度最短的树

最优二叉树(哈夫曼树)

6.5B　Huffman树及其应用

一、Huffman树

二、Huffman编码

Huffman树 → 最优二叉树

带权路径长度最短的树

Huffman编码 → 不等长编码

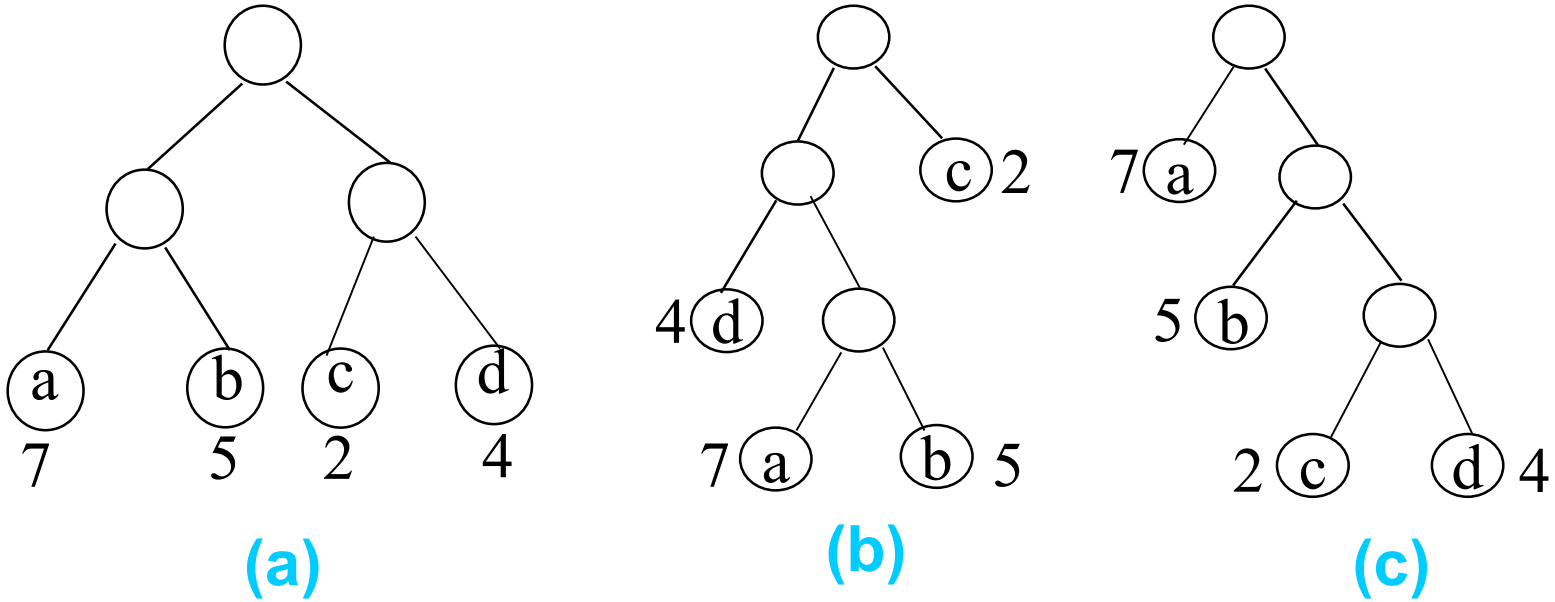
是通信中最经典的压缩编码

树的带权路径长度如何计算？

$$WPL = \sum_{k=1}^{n} w_k l_k$$

树中所有叶子结点的带权路径长度之和

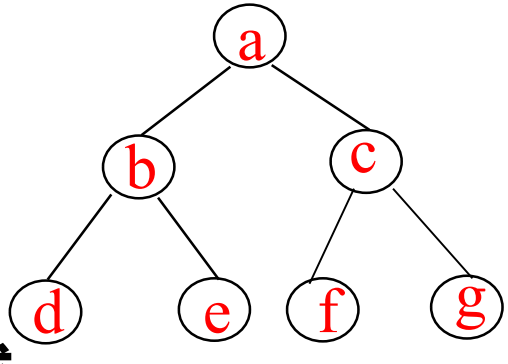
经典之例：



WPL= 36　　WPL= 46　　WPL= 35

Huffman树是 *WPL* 最小的树

一、Huffman树（最优二叉树）



若干术语：

路　　径：**由一结点到另一结点间的分支所构成。**

路径长度：**路径上的分支数目。 例如：a→e的路径长度 = 2**

树的路径长度：**从树根到每一结点的路径长度之和。树长度 = 10**

带权路径长度：**结点到根的路径长度与结点上权的乘积（WPL）**

Weighted Path Length

树的带权路径长度：**即树中所有叶子结点的带权路径长度之和**

Huffman树：**带权路径长度最小的树。**

Huffman常译为赫夫曼、霍夫曼、哈夫曼、胡夫曼等

1. 构造Huffman树的基本思想：

WPL最小的树

权值大的结点用短路径，权值小的结点用长路径。

讨论：Huffman树有什么用？

最小冗余编码、信息高效传输

例：设有4个字符d,i,a,n，出现的频度分别为7,5,2,4，怎样编码才能使它们组成的报文在网络中传得最快？

法1：等长编码（如二进制编码）

令d=00，i=01，a=10，n=11，则：

$WPL_1 = 2bit \times (7+5+2+4) = 36$

频度高的信息用短码，低的用长码，传输效率肯定高！

法2：不等长编码（如Huffman编码）

令d=0;i=10,a=110,n=111，则：

$WPL_2 = 1bit \times 7 + 2bit \times 5 + 3bit \times (2+4) = 35$

明确：要实现Huffman编码，就要先构造Huffman树

2. 构造Huffman树的步骤（即Huffman算法）：

(1) 由给定的 n 个权值$\{ w_1, w_2, \ldots, w_n \}$构成$n$棵二叉树的集合$F = \{ T_1, T_2, \ldots, T_n \}$ （即森林），其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树均空。

(2) 在F 中选取两棵根结点权值最小的树 做为左右子树构造一棵新的二叉树，且让新二叉树根结点的权值等于其左右子树的根结点权值之和。

(3) 在F 中删去这两棵树，同时将新得到的二叉树加入 F中。

(4) 重复(2) 和(3) , 直到 F 只含一棵树为止。这棵树便是Huffman树。

总之，每次合并当前值最小的两个权。
(此树特征：没有度为1的结点)

怎样证明它就是WPL最小的最优二叉树？参考《信源编码》

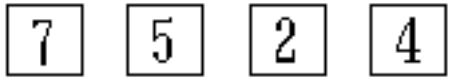
具体操作步骤：

step1：对权值进行合并、删除与替换

——在权值集合{7,5,2,4}中，总是合并当前值最小的两个权

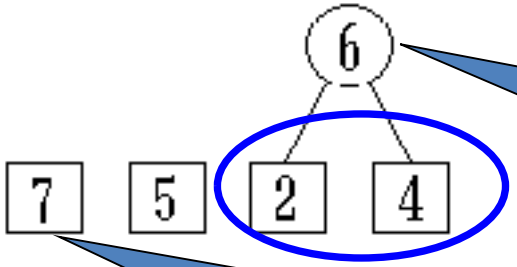
a. 初始

F：{7}{5}{2}{4}



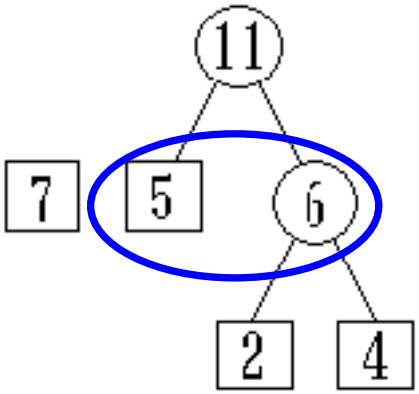
b. 合并{2} {4}

F：{7}{5}{6}



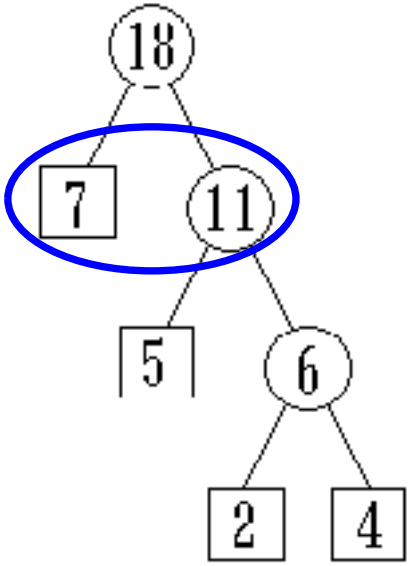
c. 合并{5} {6}

F：{7}{11}



d. 合并{7} {11}

F：{18}



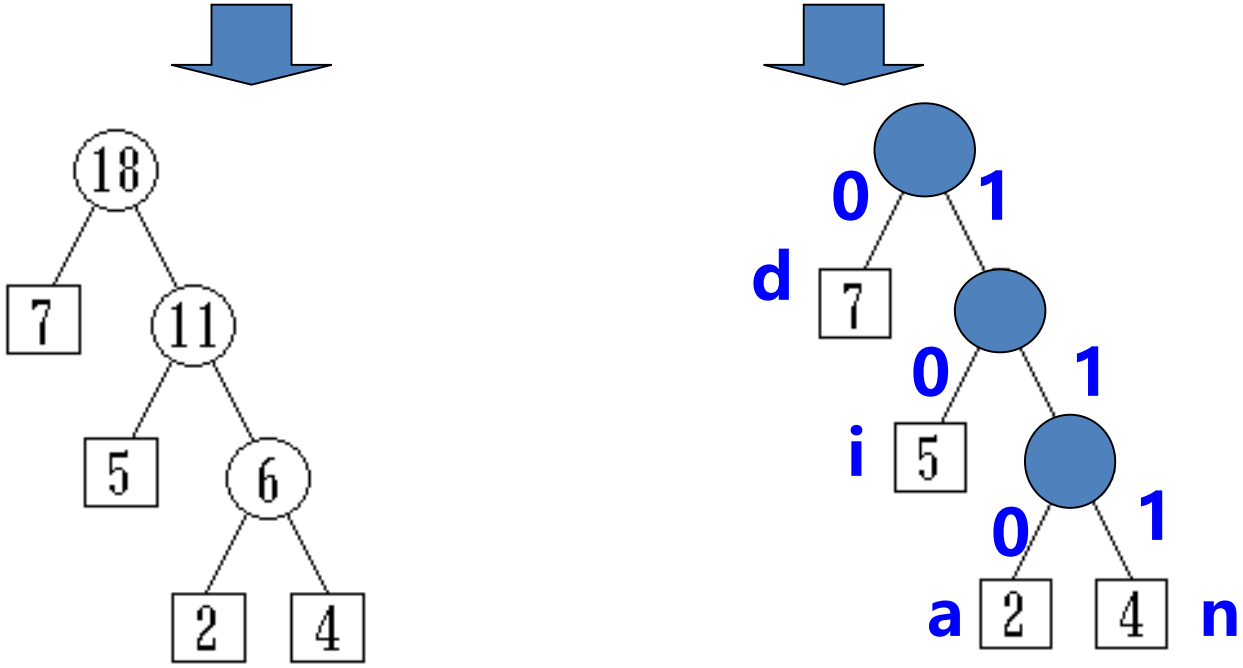
圆框表示内结点（合并后的权值）

方框表示外结点（叶子，字符）

谁左谁右？不规定就不会惟一

step2：按左“0”右“1”对Huffman树的所有分支编号

——将 Huffman树 与 Huffman编码 挂钩



Huffman编码结果：d=0，i=10，a=110，n=111

WPL=1bit×7＋2bit×5+3bit(2+4)=35（小于等长码的WPL=36）

特征：每一码不会是另一码的前缀，译码时可惟一复原

Huffman编码也称为前缀码